University of British Columbia, Vancouver
Department of Computer Science

CPSC 304 Project Cover Page

Milestone #: 4

Date: April 2, 2026

Group Number: 107

Name	Student Number	Alias (Userid)	Preferred E-mail Address
Justin Hoang	94126356	t3v8j	Just.hoang@outlook.com
Hao Hu	12260022	t9s7z	helenhaohu@gmail.com
Angel Shen	98425549	i0n5o	an.shen222@gmail.com

By typing our names and student numbers in the above table, we certify that the work in the attached assignment was performed solely by those whose names and student IDs are included above.
In addition, we indicate that we are fully aware of the rules and consequences of plagiarism, as set forth by the Department of Computer Science and the University of British Columbia

University of British Columbia, Vancouver
Department of Computer Science

2. Summary:
Our project allows users to create their own hand gesture analysis model for gesture interpretation. Users can create custom gestures for their model by writing a name and uploading videos to train their model for that gesture. They can then select the model they would like to use to analyze a video. After the analysis is done they'll be shown a transcript of the gestures the model predicted from the video.

2.5 Schema Changes:
Our schema mostly stayed the same. However, we ran into an error where the number of vectors we were using to represent gestures exceeded our 255 varchar limit. This also happened with our machine learning model and transcript. For our gestures and transcript, we chose to store them as a JSON file instead, as it made it easier and faster for the model to read and interpret data. The transcript we kept as a varchar, as we wanted to keep it as close to the original schema as possible, but we extended the limit to 1000 instead of 255. We also changed it so that one user was able to create multiple dictionaries (Definition in our schema), as limiting the freedom of the user to create multiple models felt unnecessary and made testing more difficult as well.

3. Visual Representation
CPSC 304 Visual Representation - Sheet1.pdf

4. List of all SQL queries 1-6 (SQL file and line number(s))
All SQL queries can be found in demotable_queries.sql or in the combined_sql.sql file! Each query also has their respective Python queries in db.py.

1. **lines 67-76**

```
--INSERT QUERY: add a new calibrated definition for a user
        INSERT INTO Calibrated_Definition (def_id, user_id, gesture, def_name, description)
        VALUES (
            :input_def_id,
            :input_user_id,
            :input_gesture,
            :input_def_name,
            :input_description
        );
        COMMIT;
```

2. **lines 14-21**

```
--UPDATE QUERY: changes the name and email of the tuple that has user_id=1
        UPDATE Calibrated_User
        SET
            name = :input_name,
            email = :input_email
        WHERE
            user_id = :input_user_id;
```

```
COMMIT;
```

3. **lines 31-35**

```
-- DELETE QUERY: delete an analyzed frame by recording and frame id
DELETE FROM Contained_Analyzed_Frame1
WHERE recording_id = :input_recording_id
AND frame_id = :input_frame_id;
COMMIT;
```

4. **lines 49-55**

```
--SELECTION QUERY: find all recordings created by a specific user
  --This is dynamically generated by our Python backend!! the function is below this query
SELECT recording_id, recording_name, fps, recording_date
FROM Created_Documented_Recording
WHERE 1=1
 AND fps > :val_0
 OR recording_name = :val_1;
```

 # python code below lines 159-179

```
def insert_calibrated_definition(connection, def_id, user_id, gesture, def_name,
description):
    cursor = connection.cursor()
    query = """
        INSERT INTO Calibrated_Definition (def_id, user_id, gesture, def_name,
description)
        VALUES (:input_def_id, :input_user_id, :input_gesture, :input_def_name,
:input_description)
    """
    bind_vars = {
        "input_def_id": def_id,
        "input_user_id": user_id,
        "input_gesture": gesture,
        "input_def_name": def_name,
        "input_description": description
    }
    try:
        cursor.execute(query, bind_vars)
        connection.commit()
        return "Insert successful."
    except oracledb.DatabaseError as e:
        error, = e.args
        return f"Insert failed. Database error: {error.code}"
```

5. **lines 37-41**

--PROJECTION: allows the user to select any number of attributes from their user profile to view
--This is dynamically generated by our Python backend!! the function is below this query

```
SELECT name, email
FROM Calibrated_User
WHERE user_id = :input_user_id;
```

 # python code below lines 70-95

```
def view_user_attributes(connection, user_id, selected_columns):
    """
    selected_columns example: ["name", "email"]
    """
    if not selected_columns:
        return "No columns selected."

    ALLOWED_COLS = {"user_id", "name", "email", "created_at"}
    safe_columns = []

    for col in selected_columns:
        clean_col = str(col).lower()
        if clean_col in ALLOWED_COLS:
            safe_columns.append(clean_col)
        else:
            raise ValueError(f"Security Alert: Invalid column requested -> {col}")

    cursor = connection.cursor()
    columns_string = ", ".join(safe_columns)

    query = f"SELECT {columns_string} FROM Calibrated_User WHERE user_id =
:input_user_id"
    bind_vars = {"input_user_id": user_id}

    cursor.execute(query, bind_vars)
    return cursor.fetchall()
```

6. **lines 1-12**

--JOIN QUERY: find translated words in a specific transcript + their confidence scores

```
SELECT
    TW1.instance_id,
    TW5.translation,
    TW4.translation_confidence,
    TW1.transcript_id
FROM Translated_Word_1 TW1
JOIN Translated_Word_5 TW5
    ON TW1.instance_id = TW5.instance_id
JOIN Translated_Word_4 TW4
    ON TW1.instance_id = TW4.instance_id
```

```
    WHERE TW1.transcript_id = :transcript_id;
```

5. List of all SQL queries 7-10 (SQL file and line number(s)) with explanation

All SQL queries can be found in demotable_queries.sql or in the combined_sql.sql file! Each query also has their respective Python queries in db.py.

7. **lines 57-65**

```
--GROUP BY QUERY: count how many transcripts each user has saved
    SELECT user_id, AVG(fps) AS avg_fps
    FROM Created_Documented_Recording
    GROUP BY user_id
    HAVING AVG(fps) >= ALL (
      SELECT AVG(fps)
      FROM Created_Documented_Recording
      GROUP BY user_id
    );
```

8. **lines 43-47**

```
--HAVING: checks if the selected user has a name documented
    SELECT user_id, COUNT(recording_id) AS total_recordings
    FROM Created_Documented_Recording
    GROUP BY user_id
    HAVING COUNT(recording_id) > :input_min_recordings;
```

9.**lines 23-29**

```
--NESTED AGGREGATION: find transcripts whose word count > the average word count of all transcripts
    SELECT transcript_id, word_count
    FROM Documented_Saved_Transcript_4
    WHERE word_count > (
      SELECT AVG(word_count)
      FROM Documented_Saved_Transcript_4
    );
```

10. **lines 78-92**

```
--DIVISION QUERY: find users who have saved transcripts in every language currently stored
    SELECT CU.user_id, CU.name
    FROM Calibrated_User CU
    WHERE NOT EXISTS (
      SELECT DS6.language
      FROM Documented_Saved_Transcript_6 DS6
      WHERE NOT EXISTS (
        SELECT 1
        FROM Documented_Saved_Transcript_2 DS2
        JOIN Documented_Saved_Transcript_6 DS6_USER
          ON DS2.transcript_id = DS6_USER.transcript_id
```

```
        WHERE DS2.user_id = CU.user_id
          AND DS6_USER.language = DS6.language
    )
);
```

6. Code-notice

Although we followed the timeline closely, some areas of the code were naturally worked on by people other than those listed in the timeline for many reasons. For example, originally Justin was going to also work on the backend machine learning components; however, since he was responsible for setting up the project (connecting our backend, Python, to the frontend by using routers/fastapi) and it ended up being a very large amount of work, we naturally redistributed the load of the code to make the distribution more equal. As a result, Angel primarily worked on the machine learning components.

To demonstrate these changes, **here is a summary of who is responsible for which parts of the project**:
Justin:
UI Frontend, Setting up project
Code files: All demotable files, Scripts folder, utils folder, public folder, main.py, db.py, and everything in frontend (excluding the files listed under Hao and Angel)

Hao:
Processing frames from videos, User login
Code files: translator_router.py, auth_router.py, capture.py, store.py, auth_service.py, authApi.js, AuthWorkspace.jsx, SignLanguageApi.js, login aspect of App.jsx

Angel:
Analysis of frames, Machine Learning
Code files: preprocess.py, analysis_store.py, hand_landmarker.py, ml_router.py, ml_model.py, ml_service.py, mlApi.js, MlWorkspace.jsx

All of us worked on the database and the SQL! This is also an overview, so some people may have worked on code files that another person was largely responsible for - this especially goes for the UI, as that was a lot more collaborative. We chose to split things up so distinctly this way as it was easier to have people separately code in chunks rather than code at the same time, having to deal with git mismatches.

7. AI-Declaration:

For this milestone, **WE USED AI ASSISTANCE** and asked for help learning about how to use OpenCV alongside MediaPipe for video processing. Specifically, we wanted to know if MediaPipe's included Hand Landmarker tool would be simple enough to not only program but also understand in the time we were given to code our project. We also used Gemini for different approaches to using MediaPipe, which would best fit our existing schemas and structure, and we ended up using their recommended method of taking in hand mappings as vectors and storing them in JSON files.
https://drive.google.com/file/d/1PcHfRLAfrW3L2FtqCREO2N1f-P_SRmJ8/view?usp=sharing

University of British Columbia, Vancouver
Department of Computer Science

(From the previous milestone, but we still used it throughout this milestone)
Due to the machine learning and motion sensor components of our project, Python was the most practical choice for the backend. Most of our experience with machine learning libraries comes from Python, particularly from CPSC 330. We also have little knowledge of coding machine learning models from scratch since we haven't taken CPSC310, so Python felt like the only real choice. However, none of our team members had experience connecting a Node.js and JavaScript project to a Python backend. To help with the initial setup, **WE USED AI ASSISTANCE** (ChatGPT) to help guide our project structure and properly connect Python, React, FastAPI, and the Oracle database.

This is a link to a pdf of the full conversation:
https://drive.google.com/file/d/1Ds6iV9m3b7UdvfeMEFbSJ1IUVSAdA9yr/view?usp=sharing

The following pages also include screenshots of the full ChatGPT conversation.

ChatGPT Free offer ×

Right now I have a project that uses javascript and node.js as a frontend. I want to use python with FastAPI and react. Can you give advice on how I should complete the transition? I want a backend in python with FastAPI and a frontend with react. For now I want a simple vite + react frontend.

Transitioning from a **Node.js + JavaScript frontend** to a **Python FastAPI backend + React frontend (Vite)** is a very common modernization path. The cleanest way is to **separate backend and frontend into two independent apps** that communicate via an API. Below is a practical transition plan that minimizes breakage and lets you migrate gradually. 🚀

---

1. Target Architecture

Your final structure should look like this:

```
project-root/
│
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   ├── routers/
│   │   ├── models/
│   │   └── services/
│   ├── requirements.txt
│   └── venv/
│
├── frontend/
│   ├── index.html
│   ├── package.json
│   ├── vite.config.js
│   └── src/
│       ├── App.jsx
│       ├── main.jsx
│       └── components/
```

```
└── docker-compose.yml (optional)
```

Responsibilities

Layer	Technology	Responsibility
Backend	FastAPI	API + business logic
Frontend	React (Vite)	UI
Communication	REST / JSON	fetch / axios

---

2. Create the FastAPI Backend

First build the backend **independently**.

Install dependencies

Bash

```bash
cd backend

python -m venv venv
source venv/bin/activate  # linux/mac
venv\Scripts\activate     # windows

pip install fastapi uvicorn python-multipart
```

Create:

backend/app/main.py

Python Run

```python
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

origins = [
    "http://localhost:5173",
```

```
]

app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.get("/")
def root():
    return {"message": "FastAPI backend running"}

@app.get("/api/data")
def get_data():
    return {"data": ["apple", "banana", "orange"]}
```

Run backend:

```bash
uvicorn app.main:app --reload
```

Server:

```
http://localhost:8000
```

Docs automatically available:

```
http://localhost:8000/docs
```

---

3. Create the React + Vite Frontend

From the root:

```bash
npm create vite@latest frontend
```

Choose:

```
React
JavaScript
```

Install dependencies:

```bash
cd frontend
npm install
```

Run:

```bash
npm run dev
```

Frontend:

```
http://localhost:5173
```

---

4. Connect React to FastAPI

Edit:

```
frontend/src/App.jsx
```

```javascript
import { useEffect, useState } from "react"

function App() {
  const [data, setData] = useState([])

  useEffect(() => {
    fetch("http://localhost:8000/api/data")
      .then(res => res.json())
      .then(data => setData(data.data))
  }, [])

  return (
    <>
```

```
      <h1>FastAPI + React</h1>

      <ul>
        {data.map((item, i) => (
          <li key={i}>{item}</li>
        ))}
      </ul>
    </>
  )
}

export default App
```

What is this?

Now React fetches data from FastAPI. 🎉

---

5. Recommended FastAPI Project Structure

As your backend grows, use this structure:

```
backend/app/

main.py
dependencies.py

routers/
    users.py
    auth.py
    stocks.py

models/
    user.py
    stock.py

schemas/
    user.py
    stock.py

services/
    user_service.py
    stock_service.py

database.py
config.py
```

Example router:

Python Run

```python
from fastapi import APIRouter

router = APIRouter(prefix="/users")

@router.get("/")
def get_users():
    return ["user1", "user2"]
```

---

6. Handling Environment Variables

Frontend:

frontend/.env

```
VITE_API_URL=http://localhost:8000
```

Use it:

JavaScript

```javascript
fetch(`${import.meta.env.VITE_API_URL}/api/data`)
```

---

7. Running Both Together

Two terminals:

Terminal 1

```
cd backend
uvicorn app.main:app --reload
```

Terminal 2

```
cd frontend
npm run dev
```

---

8. Optional: Use Docker Compose

Since you already use **Docker Compose** in your RankAlpha project, you could do:

```
services:

  backend:
    build: ./backend
    ports:
      - "8000:8000"

  frontend:
    build: ./frontend
    ports:
      - "5173:5173"
```

---

9. Migration Strategy (Important)

Instead of rewriting everything at once:

Step 1

Keep Node backend running.

Step 2

Start rewriting endpoints in FastAPI.

Step 3

Point React to FastAPI endpoints.

Step 4

Delete Node backend.

This avoids breaking your frontend.

---

10. Tools I Strongly Recommend

FastAPI ecosystem:

```
pydantic
sqlalchemy
alembic
python-dotenv
httpx
```

React ecosystem:

```
axios
react-router-dom
tanstack-query
```

---

11. The Biggest Mistake to Avoid ⚠️

Do **NOT** try to serve React from FastAPI during development.

Use:

```
React dev server (Vite)
+
FastAPI API server
```

Production bundling comes later.

---

12. If You Want, I Can Also Show

- ⭐ **A complete FastAPI + React starter template**
- ⭐ **A recommended folder structure for large projects**
- ⭐ **How to add authentication (JWT)**
- ⭐ **How to connect PostgreSQL with FastAPI**

Just tell me and I can generate a **clean production-ready template**.

can you give a very basic folder structure and template, just so I can get started

Here is a **very minimal FastAPI + Vite + React template** you can start with. The goal is **simplicity**, so you can get everything running in a few minutes and expand later. 🚀

Project Folder Structure

Plain text

```
my-project/
│
├── backend/
│   ├── main.py
│   └── requirements.txt
│
├── frontend/
│   ├── index.html
│   ├── package.json
│   ├── vite.config.js
│   └── src/
│       ├── main.jsx
│       └── App.jsx
│
└── README.md
```

Two completely separate apps:

- **backend → FastAPI**
- **frontend → React (Vite)**

They communicate through **HTTP requests**.

1️⃣ Backend (FastAPI)

`backend/requirements.txt`

Plain text

```
fastapi
uvicorn
```

Install:

Bash

```
cd backend
pip install -r requirements.txt
```

---

backend/main.py

Python Run

```python
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

# allow React dev server
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.get("/")
def root():
    return {"message": "FastAPI running"}

@app.get("/api/hello")
def hello():
    return {"message": "Hello from FastAPI"}
```

Run the backend:

Bash

```
uvicorn main:app --reload
```

Backend runs at:

```
http://localhost:8000
```

Docs automatically available:

```
http://localhost:8000/docs
```

---

2 Frontend (React + Vite)

Create the frontend:

```bash
npm create vite@latest frontend
```

Choose:

```
React
JavaScript
```

Then:

```bash
cd frontend
npm install
```

`frontend/src/main.jsx`

```javascript
import React from "react"
import ReactDOM from "react-dom/client"
import App from "./App"

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
```

```
  </React.StrictMode>
)
```

---

frontend/src/App.jsx

JavaScript

```javascript
import { useEffect, useState } from "react"

function App() {
  const [message, setMessage] = useState("")

  useEffect(() => {
    fetch("http://localhost:8000/api/hello")
      .then(res => res.json())
      .then(data => setMessage(data.message))
  }, [])

  return (
    <div>
      <h1>React + FastAPI</h1>
      <p>{message}</p>
    </div>
  )
}

export default App
```

What is this?

Run frontend:

Bash

```
npm run dev
```

Frontend runs at:

```
http://localhost:5173
```

---

3 Running the Project

Open **two terminals.**

Terminal 1

```bash
cd backend
uvicorn main:app --reload
```

Terminal 2

```bash
cd frontend
npm run dev
```

Open:

```
http://localhost:5173
```

You should see:

```
React + FastAPI
Hello from FastAPI
```

---

4 What This Gives You

- ✔ Working **React frontend**
- ✔ Working **FastAPI backend**
- ✔ API communication
- ✔ CORS configured
- ✔ Minimal structure

---

5 Next Steps (Recommended)

Once this works, gradually add:

Backend

```
backend/
   main.py
   routers/
   models/
   schemas/
   services/
```

Frontend

```
frontend/src/
   components/
   pages/
   api/
```

✅ If you'd like, I can also show you a **"clean beginner production structure"** that most FastAPI + React projects use (about **10 files total**) that scales much better once the project grows.